Guide to Running the Caltech Stellar Group's ZTF Forced Photometry Pipeline

Z. Vanderbosch et al.

August 22, 2024

1 Introduction

This document outlines the instructions for running the Caltech stellar group's forced photometry pipeline.

2 Running the Pipeline

1. `ssh` into the hypernova machine and `cd` into the `/data` directory:

```
> ssh username@hypernova.caltech.edu
> cd /data
```

2. Prior to running the pipeline, you should check that you have the correct date limits set. In the `psf_phot_pm_metrics.py` script, there are two global variables near the top named `PUBLIC_DATE` and `PRIVATE_DATE`. These set the upper Julian Date limits for data that will be processed by the pipeline, where "`PUBLIC`" refers to programID=1 data and "`PRIVATE`" refers to programID=2 and 3 data. I typically set the public date to the cutoff for the most recent public data release (e.g. 2460071.5 or 2023 May 7 for DR18), and for private data I set the cutoff to the day I first start running the pipeline (e.g. 2460157.5 or 2023 August 1 for the last time I ran the pipeline during Fall of 2023). To make these edits, do the following:

```
> vi psf_phot_pm_metrics.py
...
PUBLIC_DATE = 2460071.5 # Modify these upper Julian Date limits as appropriate
PRIVATE_DATE = 2460157.5
...
```

3. The `psf_phot_pm_metrics.py` script contains the main pipeline program, but must be called once per quadrant per filter. To run the pipeline in batch mode, we have two bash scripts (`run_phot_primary.sh` and `run_phot_secondary.sh`) that iterate through all quadrant+filter combinations. These two scripts are nearly identical, but one contains the field range for the ZTF primary field grid, and the other has the field range for the secondary field grid. I typically modify the field ranges, though, so the scripts only process about 100 fields at a time.

This way if something goes wrong during processing I am more likely to catch the issue before a large number of fields are affected. To modify the field ranges, use the following commands:

```
> vi run_phot_primary.sh
...
field_start=245 # Modify to a smaller range or leave these parameters as-is
field_end=881
...

> vi run_phot_secondary.sh
...
field_start=1241 # Modify to a smaller range or leave these parameters as-is
field_end=1897
...
```

4. Create a new shell session with `screen` that can be detached later while continuing to run the pipeline:

```
> screen -S name # name can be any string you'd like to name the session with
```

5. Run the bash script to start the pipeline. Here you can optionally add on the name of a file that will be used to store logging information. By default the log file is `pipeline.log` if no argument is provided. If a log file already exists, new logging info will just be appended to it.

```
> ./run_phot_primary.sh # Sets log file to pipeline.log
OR
> ./run_phot_primary.sh example.log # Sets log file to example.log
```

6. Detach from the screen session with `ctrl+a+d` and let the pipeline run. You can re-connect to the screen session at any time with

```
> screen -r name
```

3 Checking Logs for Processing Errors

The pipeline can fail on occasion, and for that purpose there is a log file within the `/data` directory that stores information for each quadrant processed by the pipeline indicating whether processing was a success or failure. By default this file is called `pipeline.log`, though the filename can be specified when running the `run_phot_primary.sh` and `run_phot_secondary.sh` scripts. Successful quadrants will have `SUCCESS` printed next to them followed by a few lines of summary info while failed quadrants will have `FAILED` printed next to them followed by the Python exception and traceback for debugging purposes. In many cases, simply re-running the pipeline on these failed quadrants works to correct any issues encountered. For this purpose, a python script `checklog.py` can be run in the `/data` directory. This script will check the log file specified with the `-f` option (default = `pipeline.log`) for any failed quadrants and save the quadrant IDs and filters to a file named `failed_quads.txt` while also printing out the failed quads for visual inspection. To rerun the pipeline on these fields, use the `run_phot_rerun.sh` bash script, which loads in the `failed_quads.txt` file and only runs the pipeline on the listed quadrants. This time the default log file name will be `pipeline_rerun.log` to separate the re-run fields from the main log file, but again this can be specified when running the bash script. So the commands would look like:

```
> python checklog.py [-f pipeline.log]
623 13 3 r # field ccd quad filter
632 6 2 i
639 3 3 r
> ./run_phot_rerun.sh # Default log file set to pipeline_rerun.log
```

4 Transferring HDF5 Matchfiles to Schoty

Once the pipeline has finished running, the new and updated HDF5 matchfiles need to be transferred to the Schoty machine where they can be accessed by other members of the group. There is another bash script `copy_matchfiles_to_schoty.sh` that uses rsync to accomplish this task. The only modifications required for this file are setting your Schoty username

5 Calculate Zero-Point Magnitude Offsets

To get forced photometry magnitudes onto the ZTF magnitude system, a zero-point offset needs to be applied. These offsets are calculated by comparing the light curve magnitudes from the forced photometry pipeline to the single-epoch light curves available through IRSA and calculating the average magnitude difference. There is a script (`get_zp_corr.sh`) in the hypernova `/data` directory that can be used to calculate these offsets and store them within a file that can then be transferred over to schoty. Running this script on all fields is fairly time-consuming, taking several weeks for just the primary grid fields, but it likely does not to be re-run each time the pipeline is run, as the offsets are unlikely to change by much with additional data. The last time these offsets were calculated was in December/January of 2023/2024 using the DR18 forced photometry light curves. To run the script, simply do the following:

```
> vi ./get_zp_corr.sh
...
for field in {245..881} # Change range to {1246..1896} for secondary fields
...
> ./get_zp_corr.sh
```